

IAD-006 – APRENDIZADO DE MÁQUINA II

Aula 03 – Gradient Boosting para Classificação, XGBoost, Data Leakage e Balanceamento de Datasets



Prof. Dr. Arthur Henrique de Andrade Melani

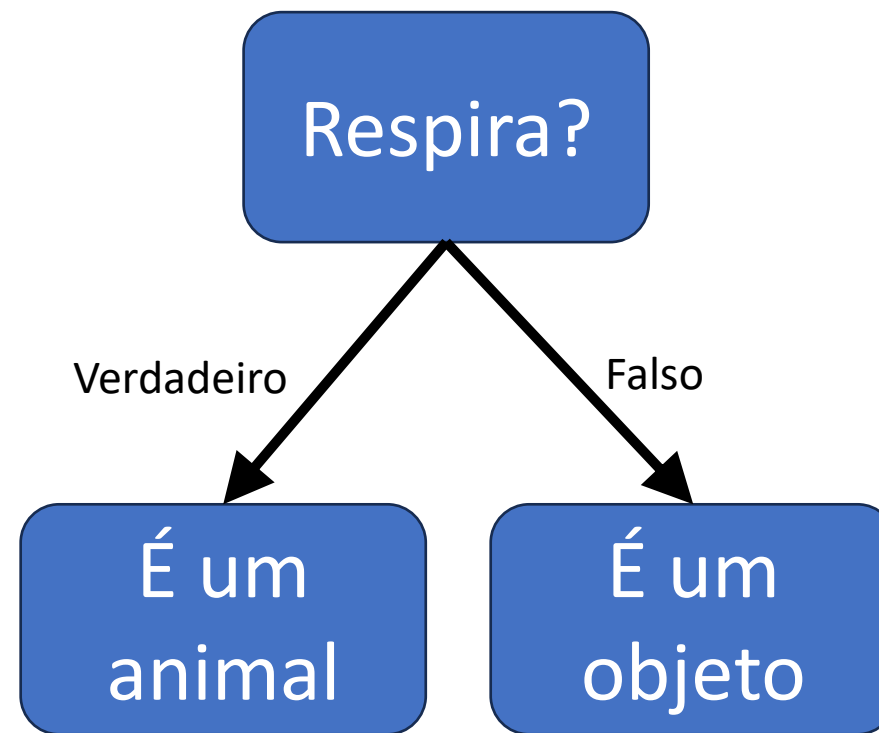
Departamento de Engenharia Mecatrônica e Sistemas Mecânicos

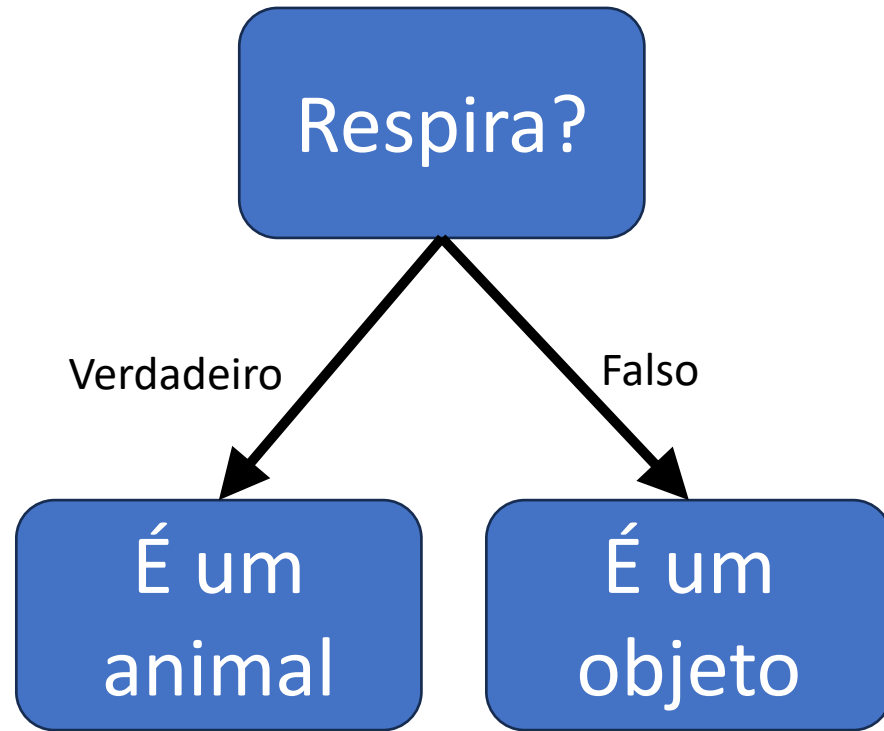
Escola Politécnica da Universidade de São Paulo

melani@usp.br

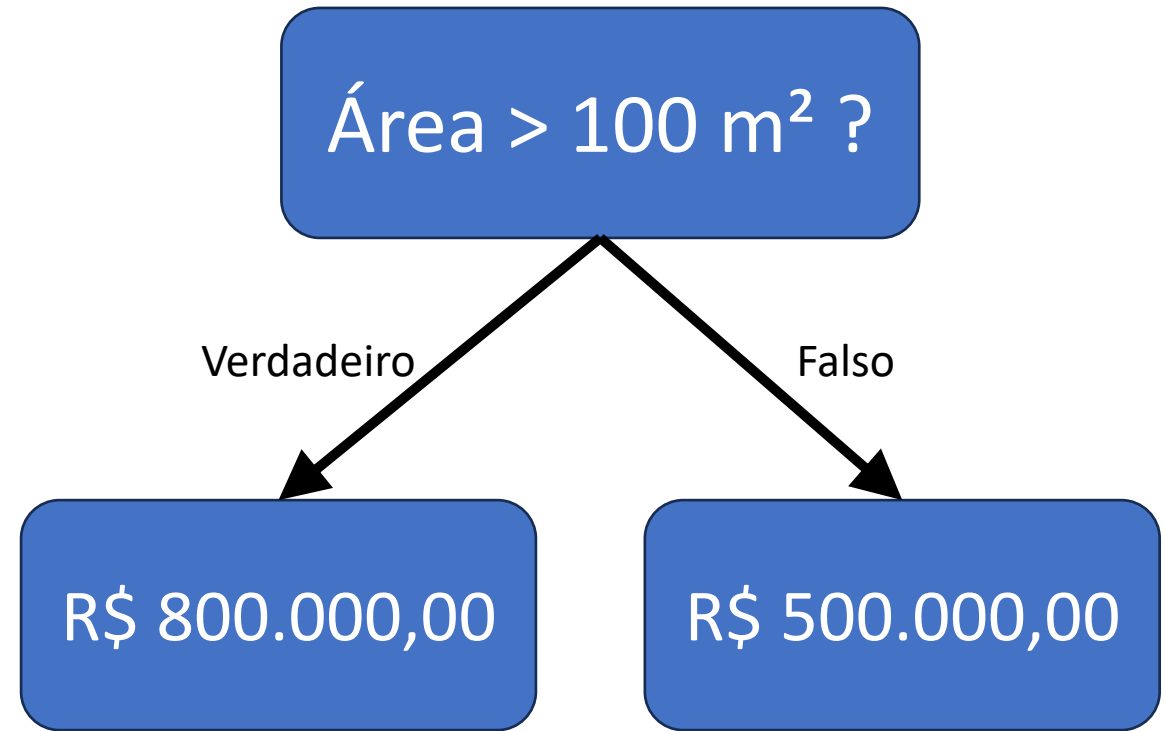
Aula 01	28/07	Árvores de Decisão
Aula 02	04/08	Métodos Ensemble
Aula 03	11/08	XGBoost e Balanceamento de Datasets
Aula 04	18/08	Trabalho 1
Aula 05	25/08	K-means, MSC e SOM
Aula 06	01/09	PCA
Aula 07	08/09	Trabalho 2
Aula 08	15/09	Máquinas de Boltzmann e Máquinas Restritas de Boltzmann
Aula 09	22/09	Autoencoders
Aula 10	29/09	Trabalho 3

- Árvore de Decisão é um modelo de aprendizado **supervisionado** utilizado para **classificação** e **regressão**.
 - Ou seja, podem prever algo categórico ou numérico!
- Representa decisões na forma de uma **estrutura hierárquica** de perguntas e respostas.
- As árvores de decisão aceitam variáveis categóricas e numéricas **sem necessidade de normalização**





- Em **classificação**, cada folha indica uma **classe** (ex: *Mamífero*, *Máquina*).



- Em **regressão**, cada folha indica um **valor numérico estimado** (ex: preço, temperatura).

Paralelo

- **Todos os modelos são treinados de forma independente**, em subconjuntos dos dados.
- A ideia é reduzir **variância** combinando modelos que erram de formas diferentes.
- As previsões são **agregadas ao final** (ex.: média, votação).
- **Erro de um modelo não afeta os outros.**
- **Exemplo:** Bagging, Random Forest
- **Aplicação:** Útil quando o modelo base tem alta variância (ex.: árvore de decisão profunda).

Sequencial

- **Os modelos são treinados um após o outro.**
- Cada novo modelo tenta **corrigir os erros do modelo anterior.**
- Foca em reduzir **viés**, aprimorando sucessivamente a predição.
- A combinação final é **ponderada**, dando mais peso a modelos mais precisos.
- **Exemplo:** Boosting, AdaBoost, Gradient Boosting, XGBoost
- **Aplicação:** Útil quando o modelo base tem alto viés (ex.: árvores rasas).

Ensembles Paralelos:

- **Averaging / Voting**
 - Combina previsões de modelos diferentes (ou iguais), por média ou votação.
- **Bagging**
 - Treina vários modelos em subconjuntos aleatórios dos dados (com reposição).
- **Random Forest**
 - Método baseado em bagging com árvores de decisão e seleção aleatória de atributos.

Ensembles Sequenciais:

- **AdaBoost**
 - Modelos fracos treinados sequencialmente, com pesos ajustados para erros.
- **Gradient Boosting**
 - Modelos são adicionados para corrigir os resíduos do modelo anterior, via gradiente.



- *Averaging* e *Voting* são as duas técnicas mais simples de comitê (ensemble)
- São técnicas do tipo Paralela, ou seja, cada modelo é **treinado de forma independente** e as **previsões são agregadas** para obter o resultado final.
- Podem ser aplicadas com **modelos iguais ou diferentes** (SVM, árvore de decisão, rede neural, etc.)

Bagging é uma técnica de ensemble que visa **reduzir a variância de modelos instáveis**, como árvores de decisão, por meio da **criação de diversos modelos treinados em subconjuntos diferentes dos dados** e da **combinação das suas previsões**.



OBS: Bootstrap aggregating = Agregação por amostragem com reposição

- **Random Forest** é uma técnica baseada em **Bagging**, que combina várias **árvores de decisão** construídas de forma independente, cada uma treinada com:
 - **Subconjuntos aleatórios de dados (bootstrap)**
 - **Subconjuntos aleatórios de features em cada divisão (split)**



AdaBoost (Adaptive Boosting) é um método de ensemble **sequencial** que combina **múltiplos classificadores fracos** (tipicamente pequenas árvores de decisão) para formar um **classificador forte e preciso**.

- **Como funciona, em essência?**

- Os modelos são treinados **um após o outro**.
- Cada novo modelo é **ajustado para corrigir os erros** cometidos pelos modelos anteriores.
- Durante o treinamento, o algoritmo **aumenta o peso das instâncias/samples mal classificadas**, forçando os próximos modelos a focarem nelas.



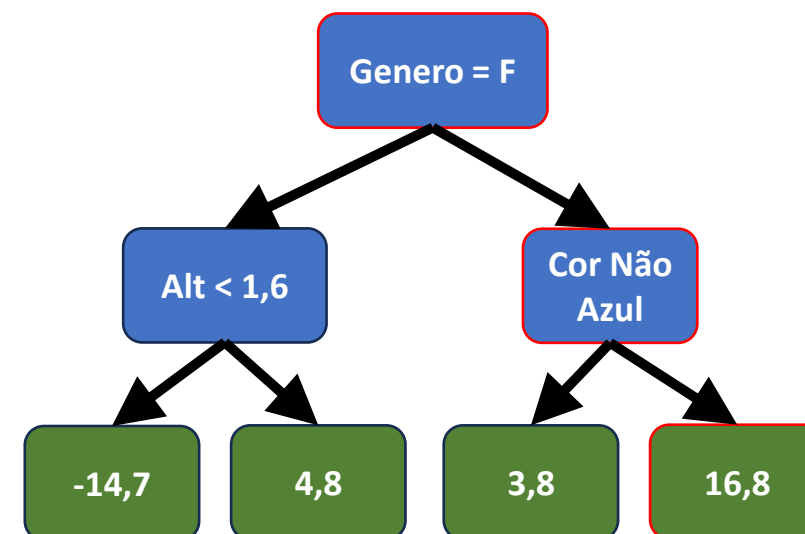
- É um método de *ensemble* **sequencial** que constrói um modelo forte adicionando, a cada etapa, um novo modelo que **corrige os erros residuais do modelo anterior**.
- A correção é feita por meio da **minimização do erro (função de perda) usando gradientes**.
- Muitos aspectos do Gradient Boosting são semelhantes ao Adaboost
 - Gradient Boosting permite que as **árvores tenham número de folhas maiores** (em geral, de 8 a 32 folhas)
- Cada preditor também é construído a partir dos erros dos preditores anteriores
- Diferente do Adaboost, cada preditor é escalado por um valor fixo.
- Neste caso, o fator usado para escalar cada preditor é chamado de **learning rate**. É definido como um hiperparâmetro.

$$\text{Valor Predito} = \text{Média} + (\text{Taxa de Aprendizado}) \times (\text{Resultado da Árvore})$$

Exemplo para a primeira amostra (considere que a taxa é 0,1):

$$\text{Valor Predito} = 71,2 + 0,1 \times 16,8 = \mathbf{72,9 \text{ kg}}$$

Altura	Cor Favorita	Gênero	Massa	Resíduo
1,6	Azul	Masculino	88	16,8
1,6	Verde	Feminino	76	4,8
1,5	Azul	Feminino	56	-15,2
1,8	Vermelho	Masculino	73	1,8
1,5	Verde	Masculino	77	5,8
1,4	Azul	Feminino	57	-14,2



- **Gradient Boosting para Classificação**
- **XGBoost**
- **Data Leakage**
- **Balanceamento de Datasets**

Gradient Boosting para Classificação

- Gradient Boosting (GB) pode ser usado tanto para regressão quanto classificação
- Para Classificação:
 - A primeira predição do GB é calculada através das fórmulas:

Gosta de Pipoca	Idade	Animal Favorito	Gosta de Cinema
Sim	12	Cachorro	Sim
Sim	87	Gato	Sim
Não	44	Cachorro	Não
Sim	19	Pássaro	Não
Não	32	Gato	Sim
Não	14	Cachorro	Sim

$$\log(odds) = \ln\left(\frac{p}{1-p}\right) = \ln\left(\frac{\overset{\text{Gostam de Cinema}}{4/6}}{\underset{\text{Não Gostam de Cinema}}{1 - 4/6}}\right) \approx 0,693 \rightarrow p_0 = \frac{1}{1 + e^{-\log(odds)}} = 0.667$$

$p_0 \approx 0,7$

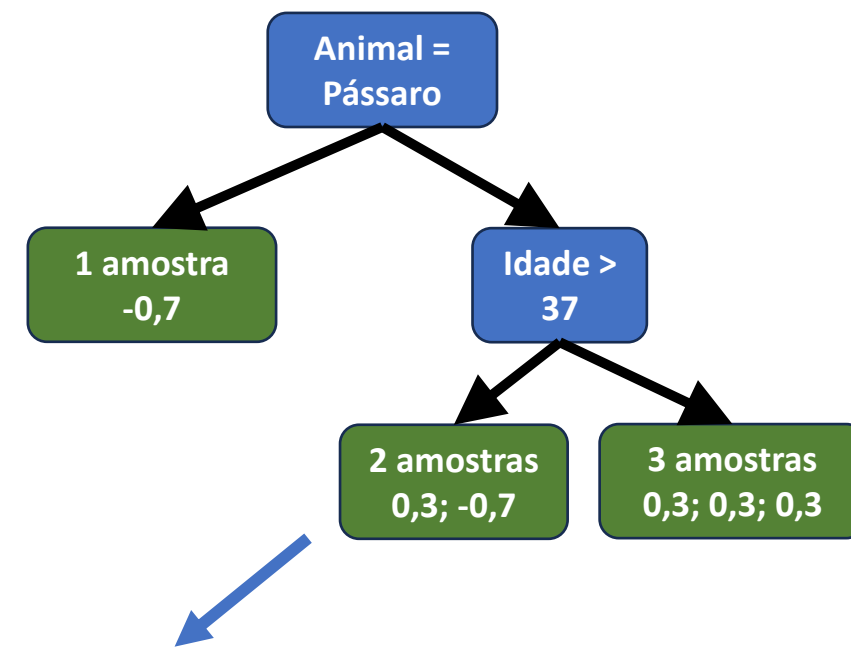
- Depois do cálculo da primeira predição, calculamos o erro da predição

$$\text{Resíduo} = \text{Valor observado} - p_0$$

Gosta de Pipoca	Idade	Animal Favorito	Gosta de Cinema	Calculando o Resíduo	Resíduo
Sim	12	Cachorro	Sim	$1 - 0,7$	0,3
Sim	87	Gato	Sim	$1 - 0,7$	0,3
Não	44	Cachorro	Não	$0 - 0,7$	-0,7
Sim	19	Pássaro	Não	$0 - 0,7$	-0,7
Não	32	Gato	Sim	$1 - 0,7$	0,3
Não	14	Cachorro	Sim	$1 - 0,7$	0,3

- A partir daí constrói-se uma árvore para prever os resíduos obtidos:

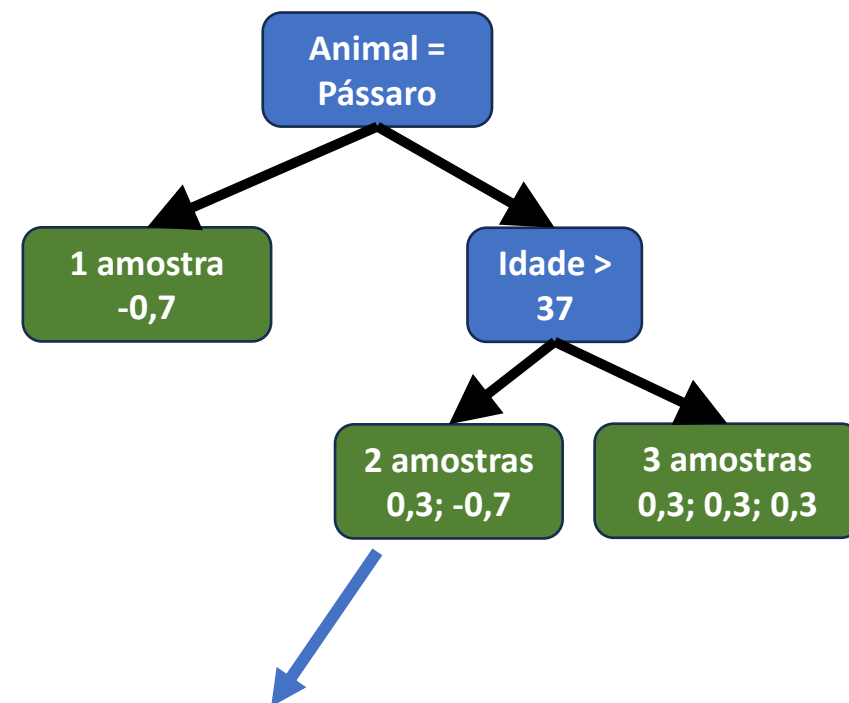
Gosta de Pipoca	Idade	Animal Favorito	Gosta de Cinema	Calculando o Resíduo	Resíduo
Sim	12	Cachorro	Sim	$1 - 0,7$	0,3
Sim	87	Gato	Sim	$1 - 0,7$	0,3
Não	44	Cachorro	Não	$0 - 0,7$	-0,7
Sim	19	Pássaro	Não	$0 - 0,7$	-0,7
Não	32	Gato	Sim	$1 - 0,7$	0,3
Não	14	Cachorro	Sim	$1 - 0,7$	0,3



É necessário calcular a predição para cada folha

- A partir daí constrói-se uma árvore para prever os resíduos obtidos:

Gosta de Pipoca	Idade	Animal Favorito	Gosta de Cinema	Calculando o Resíduo	Resíduo
Sim	12	Cachorro	Sim	$1 - 0,7$	0,3
Sim	87	Gato	Sim	$1 - 0,7$	0,3
Não	44	Cachorro	Não	$0 - 0,7$	-0,7
Sim	19	Pássaro	Não	$0 - 0,7$	-0,7
Não	32	Gato	Sim	$1 - 0,7$	0,3
Não	14	Cachorro	Sim	$1 - 0,7$	0,3

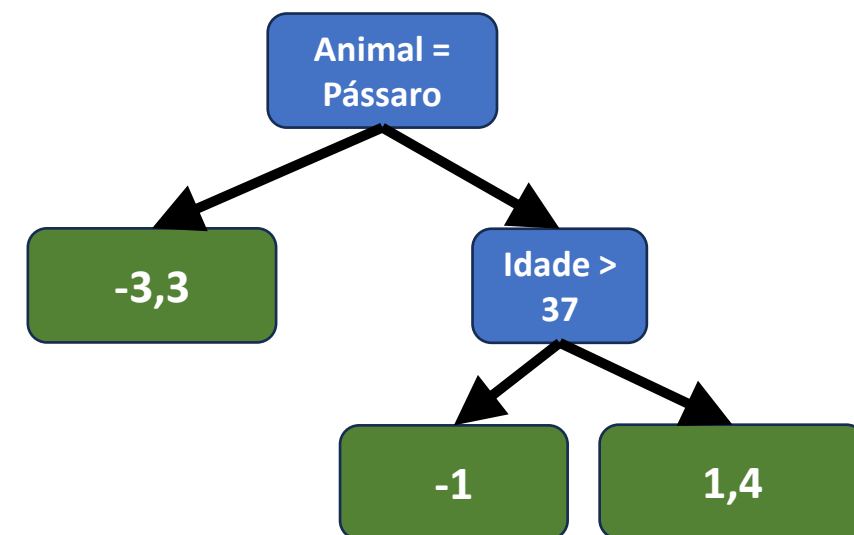


É necessário calcular a predição para cada folha:

$$\frac{\sum \text{Resíduos}}{\sum [\text{Probabilidade anterior} \times (1 - \text{Probabilidade anterior})]} = \frac{0,3 + (-0,7)}{[0,7 \times (1 - 0,7)] + [0,7 \times (1 - 0,7)]} \approx -1$$

- A partir daí constrói-se uma árvore para prever os resíduos obtidos:

Gosta de Pipoca	Idade	Animal Favorito	Gosta de Cinema	Calculando o Resíduo	Resíduo
Sim	12	Cachorro	Sim	$1 - 0,7$	0,3
Sim	87	Gato	Sim	$1 - 0,7$	0,3
Não	44	Cachorro	Não	$0 - 0,7$	-0,7
Sim	19	Pássaro	Não	$0 - 0,7$	-0,7
Não	32	Gato	Sim	$1 - 0,7$	0,3
Não	14	Cachorro	Sim	$1 - 0,7$	0,3



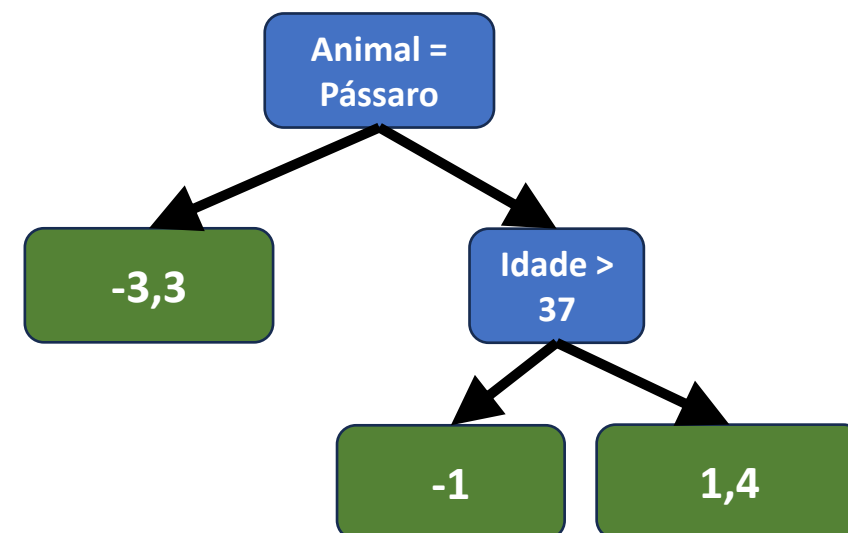
É necessário calcular a predição para cada folha:

$$\frac{\sum \text{Resíduos}}{\sum [\text{Probabilidade anterior} \times (1 - \text{Probabilidade anterior})]} = \frac{0,3 + (-0,7)}{[0,7 \times (1 - 0,7)] + [0,7 \times (1 - 0,7)]} \approx -1$$

- Com a árvore, é possível combinar seus resultados com o $\log(\text{odds})$ anteriormente obtido e a taxa de aprendizado (*learning rate*):

$$\text{Log(odds)}_{\text{novo}} = \text{log(odds)}_{\text{antigo}} + (\text{Taxa de Aprendizado}) \times (\text{Resultado da Árvore})$$

Gosta de Pipoca	Idade	Animal Favorito	Gosta de Cinema	Calculando o Resíduo	Resíduo
Sim	12	Cachorro	Sim	1 - 0,7	0,3
Sim	87	Gato	Sim	1 - 0,7	0,3
Não	44	Cachorro	Não	0 - 0,7	-0,7
Sim	19	Pássaro	Não	0 - 0,7	-0,7
Não	32	Gato	Sim	1 - 0,7	0,3
Não	14	Cachorro	Sim	1 - 0,7	0,3

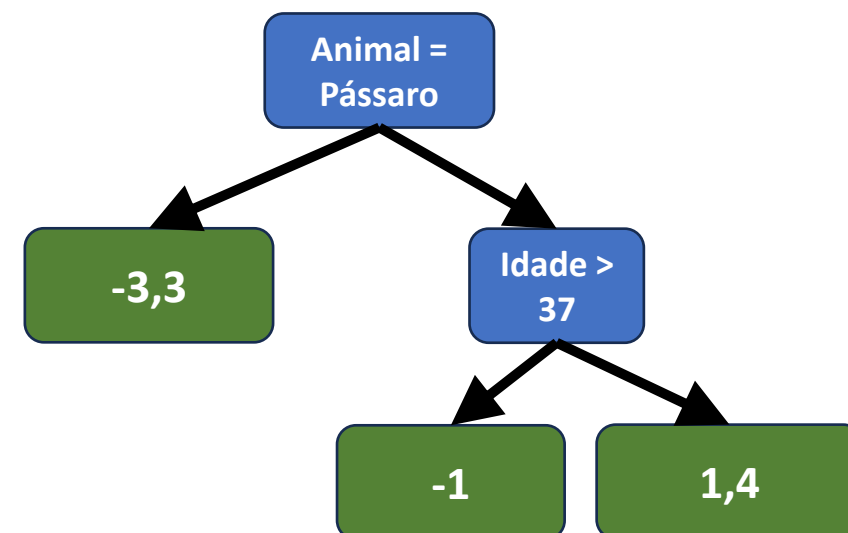


- Com a árvore, é possível combinar seus resultados com o $\log(\text{odds})$ anteriormente obtido e a taxa de aprendizado (*learning rate*):

$$\text{Log(odds)}_{\text{novo}} = \text{log(odds)}_{\text{antigo}} + (\text{Taxa de Aprendizado}) \times (\text{Resultado da Árvore})$$

Exemplo para a primeira amostra: $\text{Log(odds)}_{\text{novo}} = 0,693 + 0,8 \times 1,4 \approx 1,8$

Gosta de Pipoca	Idade	Animal Favorito	Gosta de Cinema	Calculando o Resíduo	Resíduo
Sim	12	Cachorro	Sim	$1 - 0,7$	0,3
Sim	87	Gato	Sim	$1 - 0,7$	0,3
Não	44	Cachorro	Não	$0 - 0,7$	-0,7
Sim	19	Pássaro	Não	$0 - 0,7$	-0,7
Não	32	Gato	Sim	$1 - 0,7$	0,3
Não	14	Cachorro	Sim	$1 - 0,7$	0,3



- Repete-se o processo para cada amostra e recalcula-se o resíduo:

Gosta de Pipoca	Idade	Animal Favorito	Gosta de Cinema	Probabilidade Predita	Calculando o Resíduo	Resíduo
Sim	12	Cachorro	Sim	0,9	$1 - 0,9$	0,1
Sim	87	Gato	Sim	0,5	$1 - 0,5$	0,5
Não	44	Cachorro	Não	0,5	$0 - 0,5$	-0,5
Sim	19	Vermelho	Não	0,1	$0 - 0,1$	0,0
Não	32	Gato	Sim	0,9	$1 - 0,9$	0,1
Não	14	Cachorro	Sim	0,9	$1 - 0,9$	0,1

- O processo se repete:
 - Constrói-se uma nova árvore para prever os resíduos atualizados
 - Para atualizar a predição, os resultados da nova árvore são adicionados à fórmula:

$$\text{Log(odds)}_{\text{novo}} = \text{log(odds)}_{\text{antigo}} + (\text{Taxa de Aprendizado}) \times (\text{Resultado da Árvore1}) + (\text{Taxa de Aprendizado}) \times (\text{Resultado da Árvore2}) + \dots$$

- A cada iteração, um novo modelo é treinado para **ajustar os resíduos** (erros) do modelo anterior.

XGBoost

- XGBoost significa *Extreme Gradient Boosting*.
- É uma implementação otimizada do algoritmo de Gradient Boosting.
- Desenvolvido por Tianqi Chen (2014) e amplamente adotado na indústria e em competições.
- **Objetivo:** melhorar o desempenho do Gradient Boosting tradicional, tornando-o mais **rápido, preciso e eficiente** em uso de memória.
- Funciona treinando múltiplas árvores **sequencialmente**, onde cada nova árvore corrige os erros das anteriores.
- Usado em problemas de **classificação e regressão**.

- **Regularização embutida:**
 - Reduz overfitting.
- **Poda de árvores (Tree Pruning):**
 - Evita árvores desnecessariamente complexas
- **Flexível:**
 - Suporta múltiplos tipos de modelos (árvores de decisão, regressão linear).

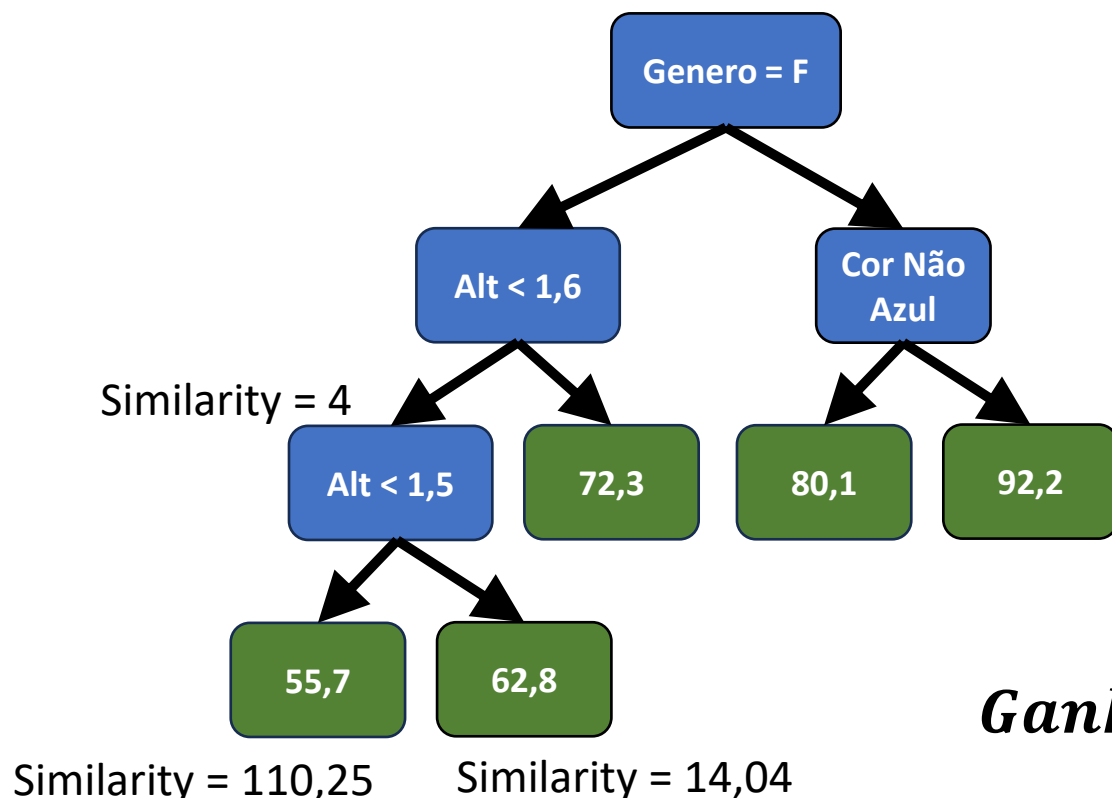
- **O que é pruning?**
 - Técnica para remover ramos da árvore de decisão que não contribuem significativamente para a melhoria do modelo.
 - Objetivo: evitar overfitting e melhorar a generalização.
- **Vantagens no XGBoost:**
 - Garante árvores mais **simples e eficientes**.
 - Reduz **tempo de predição**.
 - Mantém apenas divisões que realmente **melhoram o modelo**.



Como o XGBoost faz pruning (poda recursiva reversa):

1. Expande a árvore até a profundidade máxima permitida.
2. Calcula o ganho de informação (gain) de cada divisão.
3. Remove nós cujos ganhos são menores que o valor definido de ganho mínimo (threshold de ganho mínimo – gamma).
4. Continua a poda de baixo para cima (bottom-up) até que todos os nós restantes superem o gamma.

Exemplo visual:



Passo 1. Calcula o Similarity Score (Nota de Similaridade) para cada nó da árvore:

$$\text{Similarity Score} = \frac{(\sum \text{Resíduo}_i)^2}{\text{Quantidade de Resíduos} + \lambda}$$

Parâmetro de Regularização

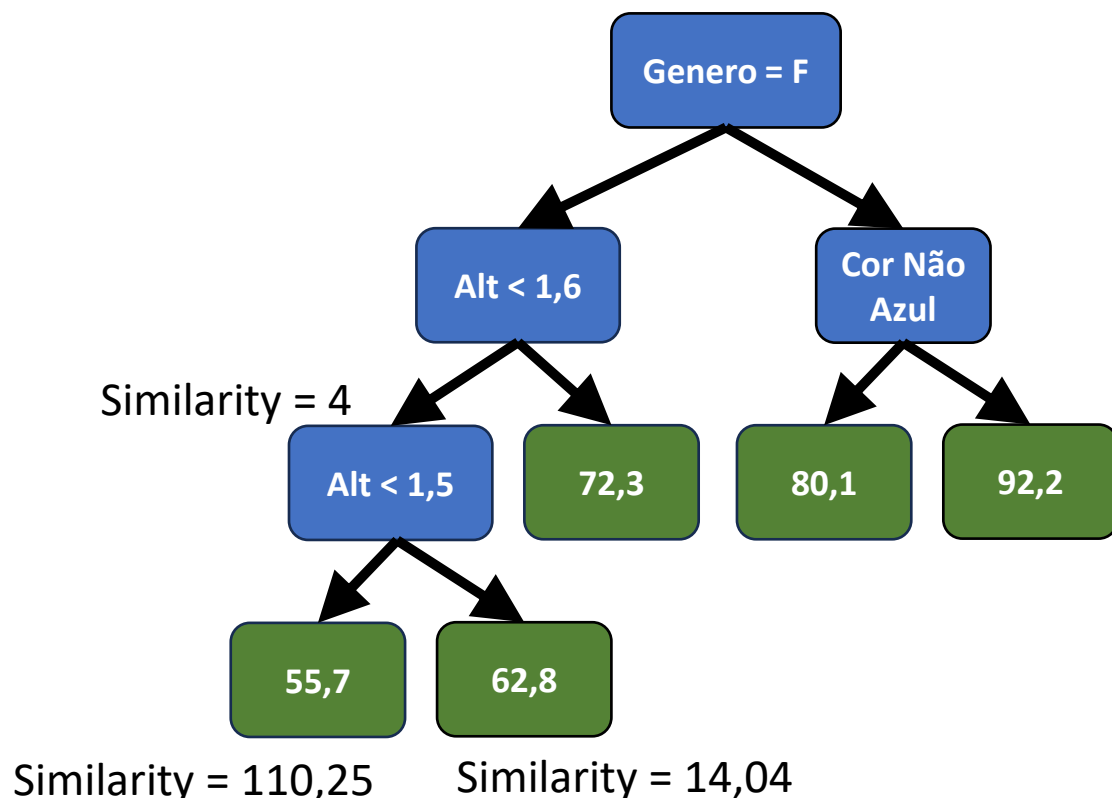
Passo 2. Calcula o Ganho:

$$\text{Ganho} = \text{Esquerda}_{\text{simil.}} + \text{Direita}_{\text{simil.}} - \text{Raiz}_{\text{simil.}}$$

$$\text{Ganho} = 110,25 + 14,04 - 4$$

$$\text{Ganho} = 120,33$$

Exemplo visual:



Passo 3. Compara Ganho com Gamma e decide se o galho deve ser podado ou não:

Ganho - $\gamma < 0 \rightarrow$ *Remover o Galho*

Ganho - $\gamma \geq 0 \rightarrow$ *Não Remover o Galho*

O **gamma** no XGBoost é um **hiperparâmetro de regularização** que define o ganho mínimo necessário para que uma divisão (split) de um nó seja aceita.

Como definir um valor para gamma?

- **Interpretação do valor de gamma:**
 - **Valor baixo (ex.: 0)** → árvores mais complexas, mais risco de overfitting.
 - **Valor alto** → árvores mais simples, mais generalização, mas risco de underfitting.
- **Como escolher:**
 - Grid Search: testar sistematicamente uma lista de valores (ex.: [0, 0.1, 0.5, 1, 5]) e medir o desempenho em validação cruzada para escolher o melhor.

- O que é Regularização?
 - Técnica para **controlar a complexidade do modelo** e evitar *overfitting*.
 - Objetivo: Melhorar a **capacidade de generalização**; evitar que o modelo aprenda **ruídos e padrões específicos demais** do conjunto de treino.
- Como Funciona:
 - Adiciona **penalizações** para desincentivar modelos muito complexos.
 - Dois tipos comuns:
 - **L1 (Lasso)**: favorece modelos mais simples, eliminando variáveis irrelevantes.
 - **L2 (Ridge)**: reduz o peso de parâmetros grandes, mas mantém todas as variáveis.

- XGBoost realiza regularização nativamente para controlar o tamanho da árvore
- Parâmetros relevantes para ajustar:
 - gamma $\rightarrow$ mínimo ganho para fazer split (Tree Pruning)
 - lambda $\rightarrow$ penalização L2
 - alpha $\rightarrow$ penalização L1 (opcional, menos usada, não será abordada).

Como definir um valor para lambda?

- Relembrando: O XGBoost usa o Similarity Score para medir a “qualidade” de um nó antes de decidir se vale a pena dividi-lo

$$\textit{Similarity Score} = \frac{(\sum \textit{Resíduo}_i)^2}{\textit{Quantidade de Resíduos} + \lambda}$$

- Como isso influencia a escolha do lambda:
 - λ pequeno:
 - Similarity score mais alto $\rightarrow$ mais splits são aceitos.
 - Modelo mais complexo, mais risco de overfitting.
 - λ grande:
 - Similarity score menor $\rightarrow$ só splits muito bons sobrevivem.
 - Modelo mais simples, mais regularizado, menos sensível a ruídos.

Como definir um valor para lambda?

- Estratégia prática para escolher: Grid Search!
- Comece com o valor padrão: $\lambda = 1$.
- Se houver overfitting (train score muito maior que validation score):
 - Aumente gradualmente: $5 \rightarrow 10 \rightarrow 20$.
- Se houver underfitting (modelo fraco no treino e validação):
 - Reduza para valores menores: $0.5 \rightarrow 0.1$.
- Sempre validar: Use cross-validation para medir impacto no score.
- **Interação com outros parâmetros:** Se λ já é alto, talvez γ possa ser menor para permitir splits mais relevantes.

Data leakage

Data leakage (vazamento de dados) ocorre quando informações que **não deveriam estar disponíveis durante o treinamento** acabam sendo utilizadas, direta ou indiretamente, pelo modelo.

Isso faz com que:

- o desempenho estimado seja **artificialmente alto**;
- o modelo pareça generalizar melhor do que realmente generaliza;
- os resultados obtidos no teste sejam **pouco confiáveis**.

Suponha que queremos padronizar uma variável: $z = (x - \mu) / \sigma$

Um procedimento incorreto seria:

- **Dataset completo $\rightarrow$ calcula μ e $\sigma \rightarrow$ padroniza o dataset $\rightarrow$ divide treino/teste**

Isso permite que informações do teste influenciem o treinamento. O procedimento mais correto é:

- **Dataset completo $\rightarrow$ divide treino/teste $\rightarrow$ calcula μ e σ apenas com treino $\rightarrow$ padroniza treino e teste**

Target leakage ocorre quando uma variável de entrada contém, direta ou indiretamente, informação sobre o **target** que não estaria disponível no momento da previsão.

Exemplo: Suponha que queremos criar um modelo para prever se um aluno será reprovado em uma disciplina:

- **Variáveis que podem ser usadas no treinamento:** notas das primeiras provas, frequência até o momento, atividades já realizadas.
- **Variáveis não que podem ser usadas no treinamento:** nota final, situação final (“reprovado”), número de faltas acumuladas até o final do semestre.

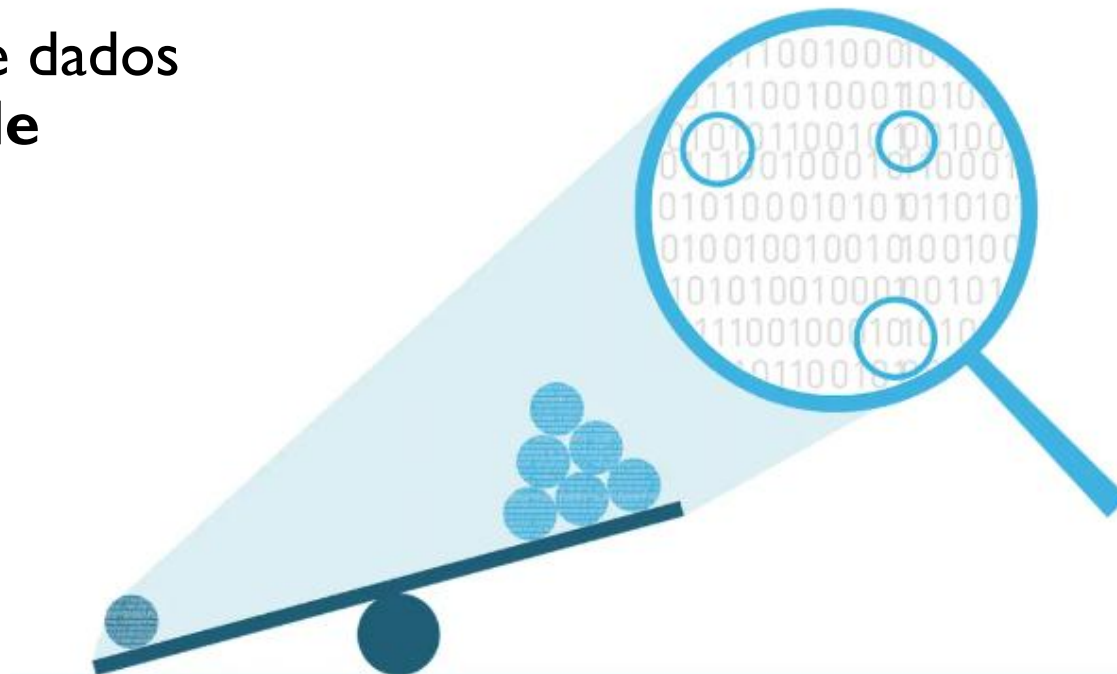
Leakage em dados temporais ocorre quando **informações do futuro** são utilizadas para treinar ou avaliar um modelo que deveria **fazer previsões sobre o futuro**.

Exemplo: Imagine que queremos criar um modelo para prever a temperature em um determinado dia usando dados históricos.

- O modelo deveria usar apenas os dados **anteriores ao instante da previsão**.

Balanceamento de Datasets

- **Definição:**
 - Ocorre quando as classes no conjunto de dados **não têm quantidades semelhantes de exemplos.**
- **Exemplo:**
 - Classificação de fraudes: 98% transações legítimas, 2% fraudulentas.
- **Por que é um problema?**
 - Modelos tendem a **favorecer a classe majoritária.**
 - Alta acurácia pode ser enganosa (acerta quase sempre a classe mais comum).
 - Difícil detectar casos raros, mas importantes.



- **Como identificar:**
 - Contar o número de exemplos por classe.
 - Visualizar a distribuição com histogramas ou gráficos de barras.
 - Regra prática: se a classe minoritária tiver menos de $\sim 10\text{--}20\%$ das amostras, atenção ao problema.
- **Métricas afetadas:**
 - Acurácia pode ser enganosa $\rightarrow$ preferir precision, recall, F1-score.
- **Exemplo numérico:**
 - Dataset com 1.000 amostras:
 - Classe A: 950 exemplos.
 - Classe B: 50 exemplos.
 - Modelo "ingênuo" que prevê sempre Classe A $\rightarrow$ acurácia = 95%, mas recall da Classe B = 0%.

- **Definição:**

- Conjunto de técnicas para ajustar a proporção de exemplos entre classes.

- **Objetivo:**

- Dar maior representatividade à classe minoritária.
- Melhorar métricas como recall e F1-score.

- **Principais abordagens:**

- Oversampling: aumentar exemplos da classe minoritária.
- Undersampling: reduzir exemplos da classe majoritária.
- Métodos híbridos: combinação de ambos.

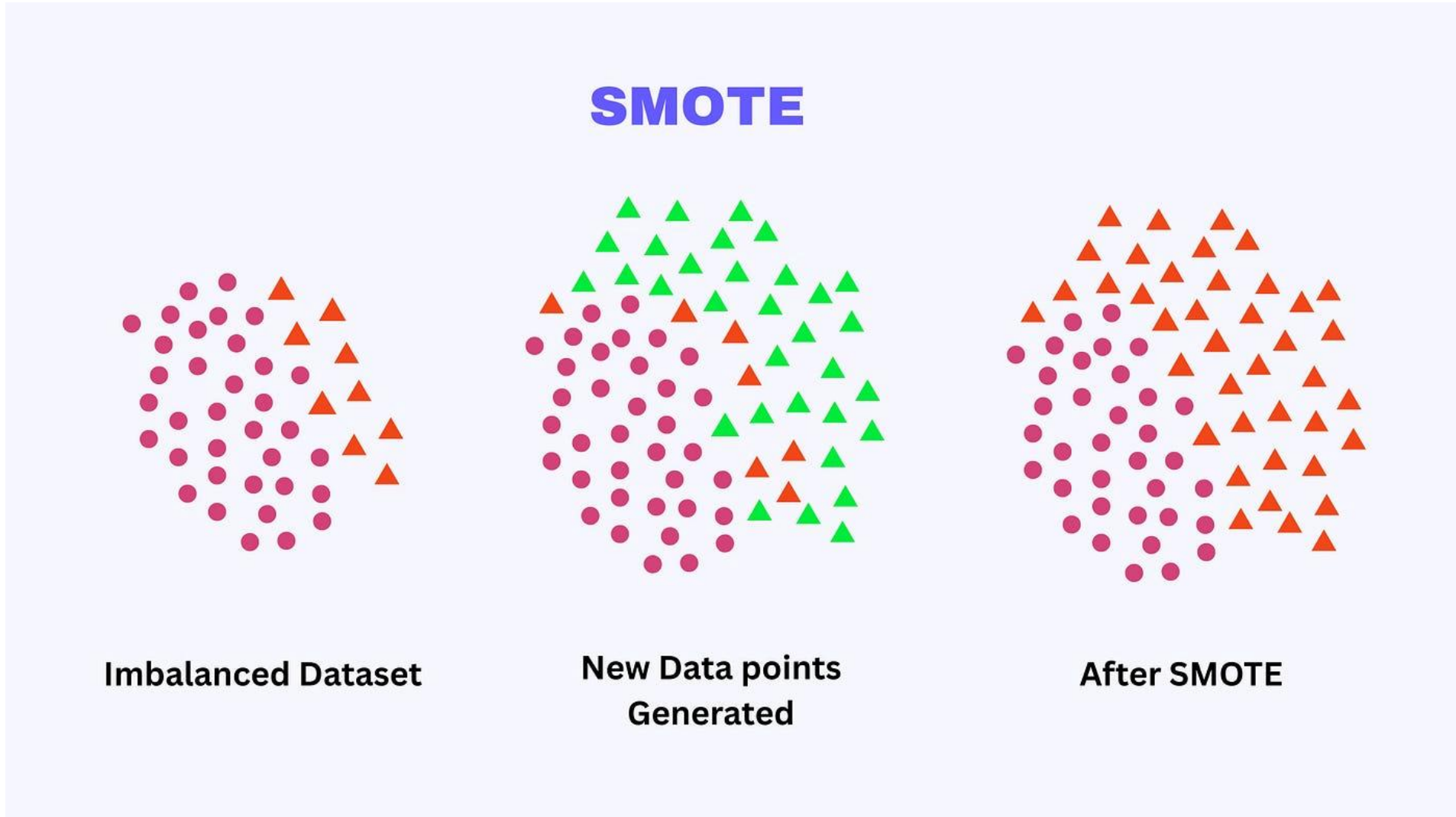
- **Benefício esperado:**

- Modelo mais justo, capaz de aprender padrões das duas classes.

O balanceamento deve ser realizado somente no conjunto de treinamento!

- O balanceamento altera artificialmente a distribuição das classes.
- O **treinamento** pode ser modificado para ajudar o modelo a aprender as classes minoritárias.
- Mas o **teste deve permanecer independente e representativo da distribuição real dos dados.**

- Estratégia para **balancear um dataset** gerando **mais amostras da classe minoritária**.
- **Random Oversampling**
 - Duplica aleatoriamente amostras da classe minoritária.
 - Simples, mas pode levar a overfitting.
- **SMOTE (Synthetic Minority Over-sampling Technique)**
 - Cria amostras sintéticas interpolando amostras existentes.
 - Reduz risco de overfitting, mas pode gerar exemplos pouco realistas.
- **ADASYN (Adaptive Synthetic Sampling)**
 - Variante do SMOTE que gera **mais amostras nas regiões de difícil classificação**.
 - Foca em áreas onde a classe minoritária está menos representada.
 - Pode melhorar a capacidade de generalização do modelo.



Passo a passo:

1. Para cada amostra da classe minoritária, encontram-se seus **k vizinhos mais próximos** (por padrão, $k=5$).
2. Escolhe-se um desses vizinhos aleatoriamente.
3. Cria-se um **novo ponto** interpolando entre a amostra original e o vizinho escolhido:

$$nova_{amostra} = amostra + aleatório(0,1) \times (vizinho - amostra)$$

4. Repete-se o processo até atingir o número desejado de novas amostras.

Exemplo:

- Suponha que será gerada uma nova amostra a partir de a_1 e a_2
- É necessário aplicar a formula abaixo:

	Feature 1	Feature 2	Feature 3	Feature 4
a_1	2	3	4	6
a_2	5	7	6	9
a_3	3	4	2	3
a_4	1.2	1.6	0.8	1.2

$$nova_{amostra} = amostra + aleatório(0,1) \times (vizinho - amostra)$$

- Número aleatório gerado: 0,4
- Para a Feature 1: $nova = a_1 + 0,4 \times (a_2 - a_1) = 2 + 0,4 \times (5 - 2) = 3,2$
- Para a Feature 2: $nova = a_1 + 0,4 \times (a_2 - a_1) = 3 + 0,4 \times (7 - 3) = 4,6$
- Para a Feature 3: $nova = a_1 + 0,4 \times (a_2 - a_1) = 4 + 0,4 \times (6 - 4) = 4,8$
- Para a Feature 4: $nova = a_1 + 0,4 \times (a_2 - a_1) = 6 + 0,4 \times (9 - 6) = 7,2$

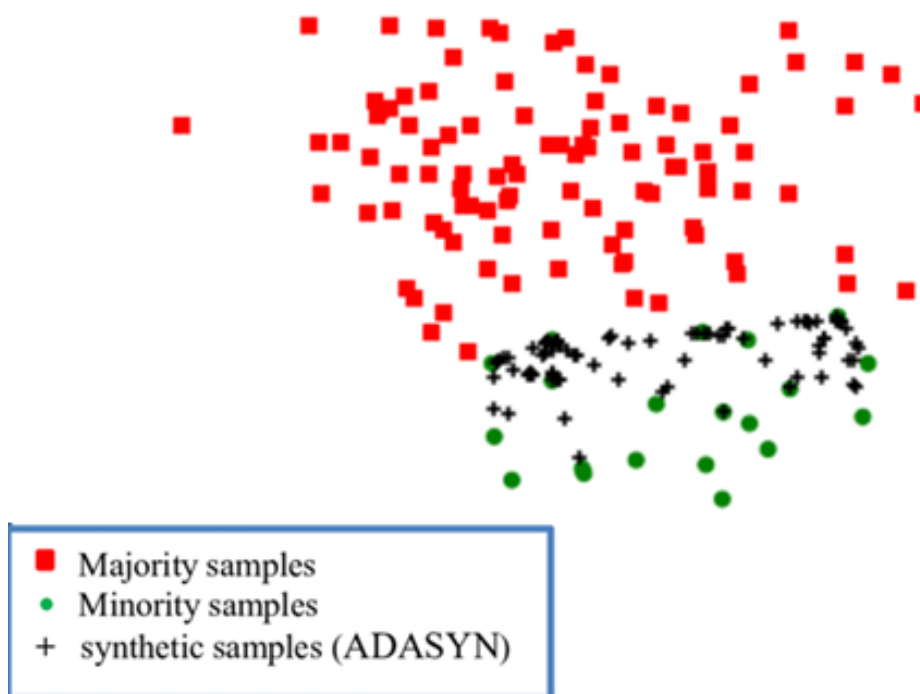
Exemplo:

- Suponha que será gerada uma nova amostra a partir de a_1 e a_2
- É necessário aplicar a formula abaixo:

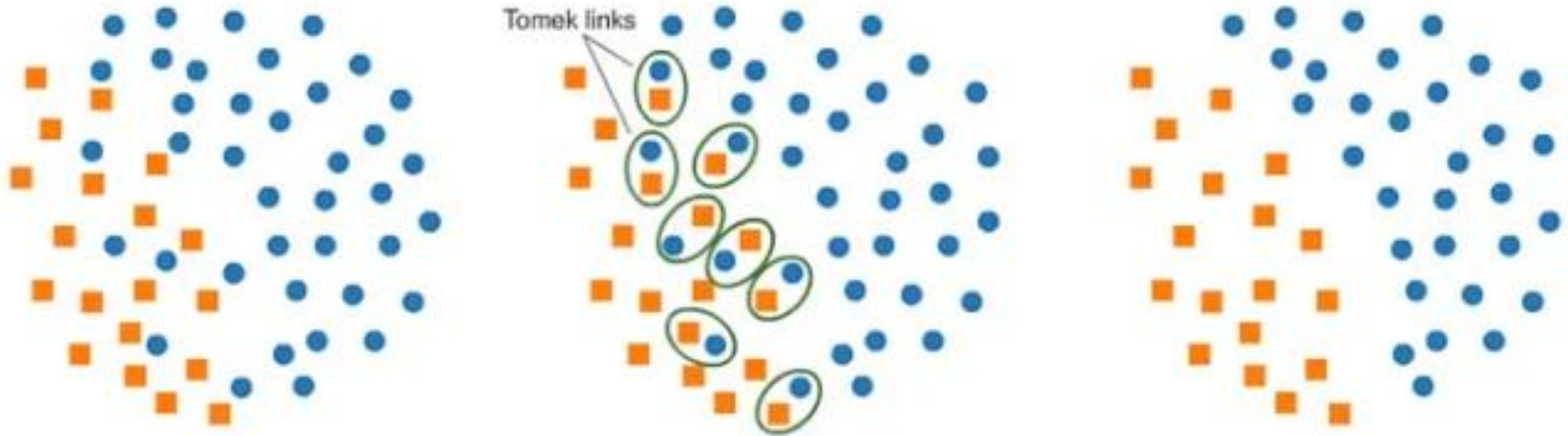
	Feature 1	Feature 2	Feature 3	Feature 4
a_1	2	3	4	6
a_2	5	7	6	9
a_3	3	4	2	3
a_4	1.2	1.6	0.8	1.2
nova amostra	3,2	4,6	4,8	7,2

- Número aleatório gerado: 0,4
- Para a Feature 1: $nova = a_1 + 0,4 \times (a_2 - a_1) = 2 + 0,4 \times (5 - 2) = 3,2$
- Para a Feature 2: $nova = a_1 + 0,4 \times (a_2 - a_1) = 3 + 0,4 \times (7 - 3) = 4,6$
- Para a Feature 3: $nova = a_1 + 0,4 \times (a_2 - a_1) = 4 + 0,4 \times (6 - 4) = 4,8$
- Para a Feature 4: $nova = a_1 + 0,4 \times (a_2 - a_1) = 6 + 0,4 \times (9 - 6) = 7,2$

	SMOTE	ADASYN
Como gera amostras	Interpola entre uma amostra minoritária e um vizinho próximo, criando pontos intermediários.	Similar ao SMOTE, mas gera mais amostras nas regiões de difícil classificação (onde há mais vizinhos da classe oposta).
Distribuição das amostras geradas	Uniforme nas regiões da classe minoritária.	Adaptativa: foca mais nas áreas com alta sobreposição ou fronteiras complexas.



- Estratégia para **balancear um dataset** diminuindo o número de amostras da classe majoritária.
- Pode ser feito por:
 - **Remoção aleatória** de amostras da classe majoritária (Random Undersampling).
 - **Seleção inteligente** das amostras mais representativas (Técnicas avançadas).
 - Tomek Links:
 - Identifica pares de amostras (uma de cada classe) que são **vizinhas mais próximas** e estão muito próximas entre si.
 - Remove a amostra da classe majoritária nesses pares, ajudando a **limpar o ruído e melhorar a separabilidade**.



Como funciona:

- Para cada amostra, encontra-se o vizinho mais próximo (usando, por exemplo, distância Euclidiana).
- Se duas amostras forem **vizinhas mútuas** e tiverem **classes diferentes**, formam um **Tomek Link**.
- Remove-se a amostra da **classe majoritária** do par → limpa a fronteira e separa melhor as classes.

O que são:

- Estratégias que combinam **aumento da classe minoritária** e **redução da classe majoritária** no mesmo processo.
- Objetivo: balancear as classes mantendo o máximo de informação útil e evitando exageros em apenas uma abordagem.

Boas práticas para definir a proporção entre oversampling e downsampling

- Se o dataset for pequeno, prefira mais oversampling (aumentar a minoria) e pouco undersampling para não perder dados.
- Se o dataset for grande, prefira mais undersampling (remover redundâncias da maioria) para reduzir custo computacional.

OBRIGADO PELA ATENÇÃO!



Prof. Dr. Arthur Henrique de Andrade Melani
Departamento de Engenharia Mecatrônica e Sistemas Mecânicos
Escola Politécnica da Universidade de São Paulo
melani@usp.br